

LESSON 303.8 · OBJECT-ORIENTED JAVA

Inheritance, Polymorphism, and Abstraction

How to share what two types have in common, and then stop caring which one you are holding. Domain: an auto insurer's fleet and property system.

SECTION ONE

Inheritance

What it is, what it's for, how to use it, and when -- in that order.

A parent class holds the state and behavior every related type will share

Every example for the rest of this lesson builds on this exact file. It is a normal, concrete class -- nothing special about it yet.

VEHICLE.JAVA — EVERY EXAMPLE AHEAD BUILDS ON THIS FILE

```
public class Vehicle {
    private final String vin, model;

    public Vehicle(String vin, String model) {
        this.vin = vin;
        this.model = model;
    }

    public String getModel() { return model; }

    public double monthlyPremium(int milesPerYear) {
        return 0.0;
    }

    public String vehicleType() { return "Vehicle"; }

    @Override
    public String toString() {
        return vehicleType() + " " + vin + " (" + model + ")";
    }
}
```

`monthlyPremium()` returns \$0.00 here because a generic vehicle isn't a real policy -- every subclass is expected to say what it actually costs to insure.

This file is the anchor. Nothing extends or implements anything ahead unless it already exists on a slide you've seen.

extends creates a child class that automatically has everything its parent has

Sedan gets vin, model, and getModel() for free. It only writes what's different.

SEDAN.JAVA

```
public class Sedan extends Vehicle {  
    public Sedan(String vin, String model) {  
        super(vin, model);  
    }  
  
    @Override  
    public double monthlyPremium(int milesPerYear) {  
        if (milesPerYear < 5000) return 45.00;  
        if (milesPerYear < 15000) return 65.00;  
        if (milesPerYear < 25000) return 90.00;  
        return 120.00;  
    }  
  
    @Override  
    public String vehicleType() { return "Sedan"; }  
}
```

Sedan declares no fields of its own -- it has vin and model because Vehicle has them. extends Vehicle is the entire relationship: everything else on this slide is what makes Sedan a Sedan.

Java allows one parent class per subclass. There is no multiple inheritance of classes -- interfaces cover that case, later.

This is what extends just saved you from writing

Without a shared parent, Sedan and Truck would each store a VIN and a model and price a monthly premium completely on their own.

Sedan.java, if Vehicle didn't exist

```
public class Sedan {
    private final String vin, model;
    public Sedan(String vin, String model) { this.vin = vin; this.model = model; }
    public double monthlyPremium(int milesPerYear) {
        if (milesPerYear < 5000) return 45.00;
        if (milesPerYear < 15000) return 65.00;
        if (milesPerYear < 25000) return 90.00;
        return 120.00;
    }
}
```

Truck.java, if Vehicle didn't exist — same shape, different numbers

```
public class Truck {
    private final String vin, model;
    public Truck(String vin, String model) { this.vin = vin; this.model = model; }
    public double monthlyPremium(int milesPerYear) {
        if (milesPerYear < 5000) return 70.00;
        if (milesPerYear < 15000) return 95.00;
        if (milesPerYear < 25000) return 130.00;
        return 170.00;
    }
}
```

Inheritance exists so the shared part is written once and the different part is written once. The real Sedan and Truck, extending Vehicle, never look like this.

super(...) reaches back to the parent's constructor

A subclass constructor's first statement is always a call to the parent's constructor, whether you write it or Java inserts it for you.

TRUCK.JAVA

```
public class Truck extends Vehicle {  
    public Truck(String vin, String model) {  
        super(vin, model);  
    }  
  
    @Override  
    public double monthlyPremium(int milesPerYear) {  
        if (milesPerYear < 5000) return 70.00;  
        if (milesPerYear < 15000) return 95.00;  
        if (milesPerYear < 25000) return 130.00;  
        return 170.00;  
    }  
  
    @Override  
    public String vehicleType() { return "Truck"; }  
}
```

`vin` and `model` are private in `Vehicle`, so `Truck` cannot set them directly. `super(vin, model)` is `Truck` asking the parent to do it.

`super(...)` must be the first line of the constructor. Leave it out and Java silently inserts a no-arg `super()` -- which fails to compile the moment the parent has no no-arg constructor of its own.

A subclass gets every non-private member of its parent for free

Sedan never declares getModel(). It has it anyway.

RUNNABLE JAVA PROGRAM

```
public class InheritedCheck {
    public static void main(String[] args) {
        Sedan s = new Sedan("V-100", "Camry");
        System.out.println(s.getModel());
    }
}
```

OUTPUT

Camry

MEMBER	INHERITED?
public / protected fields and methods	Yes, usable directly
private fields and methods	No — not visible to the subclass at all
constructors	Never. Every subclass writes its own, and its first line reaches the parent's via super(...)
static fields and methods	Yes, inherited, but never overridden — they're resolved by declared type, not runtime type

If it's private, it isn't inherited in any usable sense — it just exists on the object, invisible to the subclass's own code.

@Override replaces the parent's version, it doesn't add a second one

SemiTruck extends Truck and replaces `monthlyPremium()`, reusing Truck's version inside its own.

SEMITRUCK.JAVA

```
public class SemiTruck extends Truck {  
    public SemiTruck(String vin, String model) { super(vin, model); }  
  
    @Override  
    public double monthlyPremium(int milesPerYear) {  
        return super.monthlyPremium(milesPerYear) + 50.00;  
    }  
  
    @Override  
    public String vehicleType() { return "SemiTruck"; }  
}
```

INSIDE A MAIN() METHOD

```
Truck t = new Truck("V-200", "F-150");  
System.out.println(t.vehicleType() + " premium=" + t.monthlyPremium(12000));  
  
SemiTruck st = new SemiTruck("V-201", "Freightliner Cascadia");  
System.out.println(st.vehicleType() + " premium=" + st.monthlyPremium(12000));
```

OUTPUT

```
Truck premium=95.0  
SemiTruck premium=145.0
```

`super.monthlyPremium(milesPerYear)` calls the PARENT's version by name, so `SemiTruck` reuses `Truck`'s tiers instead of re-typing them.

A typo without @Override just compiles as a brand new, unused method

monthlypremium (lowercase p) is a different signature than monthlyPremium. Java doesn't know it was supposed to be an override.

SEMITRUCK.JAVA – THE TYPO (COMPILES FINE)

```
public class SemiTruck extends Truck {  
    public SemiTruck(String vin, String model) { super(vin, model); }  
  
    public double monthlypremium(int milesPerYear) {  
        return super.monthlyPremium(milesPerYear) + 50.00;  
    }  
}
```

INSIDE A MAIN() METHOD

```
Truck t = new SemiTruck("V-201", "Freightliner Cascadia");  
System.out.println(t.monthlyPremium(12000));
```

OUTPUT

95.0



Wrong, and silent

monthlypremium is a different signature than monthlyPremium, so SemiTruck's version of monthlyPremium is just Truck's, unchanged. The \$50 commercial surcharge never applies -- and nothing about this fails to compile or run.

This compiles and runs cleanly. The only thing wrong with it is the answer.

@Override doesn't create the override — it checks that one already happened

Add @Override to the exact same typo and the compiler catches it immediately.

SEMITRUCK.JAVA — DOES NOT COMPILE

```
public class SemiTruck extends Truck {  
    public SemiTruck(String vin, String model) {  
        super(vin, model);  
    }  
  
    @Override  
    public double monthlypremium(int milesPerYear) {  
        return super.monthlyPremium(milesPerYear) + 50.00;  
    }  
}
```

JAVAC OUTPUT

```
SemiTruck.java:6: error: method does not override or  
implement a method from a supertype  
@Override  
^  
1 error
```

Same typo, same file, one added line -- and the wrong-answer bug from the previous slide became a build failure instead of a silent \$50 loss.

@Override costs nothing and turns a silent wrong-answer bug into a compile error. Put it on every override, every time.

Override replaces a method; overload adds a second one with the same name

They sound similar and do opposite things.

	OVERRIDE	OVERLOAD
Signature	Identical to the parent's	Same name, different parameters
Where	Subclass replacing a parent method	Same class, or a subclass adding a new version
Which one runs	Decided at runtime, by the object's actual type	Decided at compile time, by the arguments you pass
@Override	Always add it — lets the compiler check your work	Never applies — there's nothing being overridden

SemiTruck's `monthlyPremium(int milesPerYear)` overrides Truck's. If SemiTruck also declared `monthlyPremium(int milesPerYear, int trailers)`, that would be an overload — a second, separate method that happens to share a name.

If the parameter list changed at all, it's an overload, not an override, and the parent's version is still there, untouched.

Reach for extends when a type genuinely IS another type, not just to save typing

Sign you need extends

Two types are the same kind of thing, and one is a specialized version of the other — a Sedan really is a Vehicle, a SemiTruck really is a Truck.

Sign you need extends

They share real fields and method bodies today, not just a label that happens to match — vin, model, and toString() are actually identical across every vehicle type.

Sign you should reconsider

You're tempted to extend a class just to reuse one method, and the two types otherwise have nothing in common. That's what interfaces are for — coming up in Section Four.

extends is a promise that a subclass IS the parent, in every situation the parent is used. If that promise doesn't hold, don't extend.

YOU TRY IT

Add a third vehicle type on your own

YOUR TURN · 5 MIN

SOLVE THIS

The fleet also insures motorcycles. Write `Motorcycle` extends `Vehicle`. Monthly premium: \$25 under 5,000 miles/year, \$35 for 5,000–14,999, \$50 for 15,000 or more. `vehicleType()` should return "Motorcycle".

One way to write it

ONE WAY TO DO IT

MOTORCYCLE.JAVA

```
public class Motorcycle extends Vehicle {
    public Motorcycle(String vin, String model) { super(vin, model); }

    @Override
    public double monthlyPremium(int milesPerYear) {
        if (milesPerYear < 5000) return 25.00;
        if (milesPerYear < 15000) return 35.00;
        return 50.00;
    }

    @Override
    public String vehicleType() { return "Motorcycle"; }
}
```

```
MOTORCYCLE M = NEW MOTORCYCLE(...); MILESPERYEAR = 10000
```

```
Motorcycle premium=35.0
```

\$25 UNDER 5,000 / same if-ladder shape as Sedan and Truck, new numbers
\$35 FOR 5,000–
14,999 / \$50 FOR
15,000+

VEHICLETYPE() = the only other required override
"MOTORCYCLE"

SECTION TWO

Polymorphism, Casting, and instanceof

What it is, what it's for, how to use it, and when -- same order, new topic.

Polymorphism means the same line of code runs a different method body depending on the object's real type

Vehicle sets the menu -- only Vehicle's methods are callable through this variable. SemiTruck decides which version of each one actually runs.

RUNNABLE JAVA PROGRAM

```
public class DeclaredRuntime {  
    public static void main(String[] args) {  
        Vehicle v = new SemiTruck("V-201", "Freightliner Cascadia");  
  
        System.out.println(v.vehicleType());  
        System.out.println(v.monthlyPremium(12000));  
    }  
}
```

OUTPUT

```
SemiTruck  
145.0
```

The declared type is `Vehicle` — the compiler only lets you call methods `Vehicle` defines. The runtime type is `SemiTruck` — that's whose overrides actually execute.

Declared type: what the compiler checks. **Runtime type:** what actually runs. They can differ any time a variable's type is a parent of the object it holds.

One loop, one method call, four different tier tables underneath

The loop never checks what kind of vehicle it's holding. It doesn't need to.

PREMIUMREPORT.JAVA – INSIDE MAIN()

```
Vehicle[] fleet = {  
    new Sedan("V-100", "Camry"),  
    new Truck("V-200", "F-150"),  
    new SemiTruck("V-201", "Freightliner Cascadia"),  
    new Sedan("V-101", "Accord")  
};  
  
double totalPremiums = 0.0;  
for (int i = 0; i < fleet.length; i++) {  
    double premium = fleet[i].monthlyPremium(12000);  
    System.out.println(fleet[i] + " $" + premium);  
    totalPremiums = totalPremiums + premium;  
}  
  
System.out.println("Total premiums: $" + totalPremiums);
```

OUTPUT

```
Sedan V-100 (Camry) $65.0  
Truck V-200 (F-150) $95.0  
SemiTruck V-201 (Freightliner Cascadia) $145.0  
Sedan V-101 (Accord) $65.0  
Total premiums: $370.0
```

`fleet[i].monthlyPremium(12000)` dispatches to a different method body each time, and the loop calling it never had to know that.

Storing a child object in a parent-typed variable is called upcasting

It happens on almost every line of the previous slide, automatically, with no cast operator written anywhere.

INSIDE A MAIN() METHOD — UPCASTING

```
Vehicle v = new SemiTruck("V-201", "Freightliner Cascadia");
```

A SemiTruck IS a Truck IS a Vehicle, so Java allows this assignment with no cast keyword at all — that's what makes it *upcasting*: moving up the family tree is always safe, so the compiler just does it for you.



Vocabulary

Upcasting: subclass reference stored as its superclass type. Always safe, always automatic, never needs a cast operator.

Every `Vehicle[]` array on the previous slide is upcasting happening in bulk -- Sedan, Truck, and SemiTruck objects, all stored as `Vehicle`.

Going the other way is called downcasting, and it can fail at runtime

The compiler only checks that the cast is plausible for the DECLARED type. Whether it actually works depends on the object underneath.

RUNNABLE JAVA PROGRAM

```
public class UpcastDowncast {  
    public static void main(String[] args) {  
        Vehicle v = new Sedan("V-100", "Camry");  
        Truck t = (Truck) v;  
        System.out.println(t);  
    }  
}
```

RUNTIME CRASH

```
Exception in thread "main" java.lang.ClassCastException:  
class Sedan cannot be cast to class Truck  
at UpcastDowncast.main(UpcastDowncast.java:4)
```

`v`'s declared type is `Vehicle`, so the compiler allows the cast to any `Vehicle` subclass, including `Truck` — that's why line 4 compiles. But the object underneath is actually a `Sedan`, and a `Sedan` is never a `Truck`, so the cast fails the instant it runs.

A cast is a promise to the compiler. If the runtime type doesn't back it up, the JVM throws `ClassCastException` instead of quietly getting it wrong.

A written-but-wrong cast fails at runtime; a missing cast fails at compile time

These look similar but are two completely different failures.

	TRUCK T = V; (NO CAST WRITTEN)	TRUCK T = (TRUCK) V; (CAST WRITTEN)
Does it compile	Never -- Java won't attempt an implicit narrowing conversion	Yes, if Truck is a plausible subclass of v's declared type
What happens at runtime	N/A -- the build never succeeds	Succeeds if the real object is a Truck; throws ClassCastException if not
Is this downcasting	No -- it's just a type error, caught before the program ever runs	Yes -- this is what downcasting means

Downcasting always needs the cast operator written out. Forgetting it isn't a shortcut -- it's a different bug entirely, and Java catches it earlier.

instanceof checks the runtime type before you commit to a cast

Pattern-matching instanceof both checks and casts in one step.

RUNNABLE JAVA PROGRAM

```
public class SafeCast {  
    public static void main(String[] args) {  
        Vehicle v = new SemiTruck("V-201", "Freightliner Cascadia");  
        if (v instanceof Truck t) {  
            System.out.println("safe downcast, premium " + t.monthlyPremium(12000));  
        }  
  
        Vehicle sedan = new Sedan("V-100", "Camry");  
        if (sedan instanceof Truck t) {  
            System.out.println("never printed");  
        } else {  
            System.out.println("sedan is not a Truck");  
        }  
    }  
}
```

OUTPUT

```
safe downcast, premium 145.0  
sedan is not a Truck
```

`v instanceof Truck t` is true and false in exactly the cases where `(Truck) v` would have succeeded or thrown — but here you get a boolean back instead of a crash, and `t` is only in scope where the check passed.

instanceof before casting turns a possible crash into a branch you control.

Downcast only when you need something the parent type doesn't expose

Stay at the parent type

If the method you need is declared on `Vehicle`, call it on the `Vehicle` reference directly — no cast needed, and overriding already picks the right body.

Downcast, guarded by `instanceof`

If you need something **ONLY** the subclass has (a `SemiTruck`-only surcharge field, say) and you're not certain what's actually in the variable.

Downcast, unguarded

Only when you just upcast it yourself two lines ago and know exactly what's in there — otherwise this is how `UpcastDowncast`'s crash happens.

Needing to downcast a lot is usually a sign the parent class is missing a method that belongs on every subtype -- which is exactly what abstraction, next, is for.

PROOF

Declared type sets the menu; runtime type cooks the meal -- proven, not guessed

v is declared as Vehicle. What actually runs when you call a method on it depends on what v really is underneath.

RUNNABLE JAVA PROGRAM

```
public class DispatchProof {  
    public static void main(String[] args) {  
        Vehicle v = new SemiTruck("V-201", "Freightliner Cascadia");  
        System.out.println("declared type: Vehicle");  
        System.out.println("runtime class: " + v.getClass().getSimpleName());  
        System.out.println("v.monthlyPremium(12000) = " + v.monthlyPremium(12000));  
    }  
}
```

OUTPUT

```
declared type: Vehicle  
runtime class: SemiTruck  
v.monthlyPremium(12000) = 145.0
```

v's TYPE never changes -- it's Vehicle on every line, and that's all the compiler ever checks. v's CLASS is what actually runs: Truck's tiered logic plus SemiTruck's own \$50 surcharge, the exact numbers from Section One's SemiTruck slide.

Two different questions, two different answers: what does the compiler allow (declared type) vs. what actually executes (runtime type).

YOU TRY IT

Write the fleet loop yourself

YOUR TURN · 6 MIN

SOLVE THIS

Build a `Vehicle[]` holding one Sedan and one Truck. Loop over it and print each vehicle's `toString()` next to its `monthlyPremium(3000)`. Do not write an `if/else` that checks which type you're holding.

One way to write it

ONE WAY TO DO IT

RUNNABLE JAVA PROGRAM

```
public class CounterPractice {  
    public static void main(String[] args) {  
        Vehicle[] items = {  
            new Sedan("V-140", "Civic"),  
            new Truck("V-260", "Silverado")  
        };  
  
        for (int i = 0; i < items.length; i++) {  
            System.out.println(items[i] + " premium=" + items[i].monthlyPremium(3000));  
        }  
    }  
}
```

OUTPUT

```
Sedan V-140 (Civic) premium=45.0  
Truck V-260 (Silverado) premium=70.0
```

No if/else was needed, and that was the point

ITEMS[I] declared type Vehicle, runtime type Sedan or Truck depending on i

ITEMS[I].MONTHLYPRICE each method dispatches to that object's own override

ITEMS[I] (IN A PRINTLN) calls toString(), also overridden nowhere here, inherited from Vehicle

NO INSTANCE OF ANYWHERE the loop body is identical no matter how many vehicle types exist

A polymorphic loop's body doesn't grow when you add a new subclass. That's the entire economic case for inheritance and overriding.

What you can defend so far

extends

IS-A. One parent, `super(...)` first, and `@Override` on every replacement

Two types

declared sets the menu and is checked by the compiler; runtime cooks the meal

upcasting / downcasting

upcasting is automatic and always safe; downcasting needs a cast operator and can throw `ClassCastException`

instanceof

checks the runtime type before a downcast, so a mismatch becomes a branch instead of a crash

Next: what happens when a class has no sensible default at all.

SECTION THREE

Abstraction

What it is, what it's for, how to use it, and when.

An abstract class can declare a method with no body, forcing every subclass to write its own

Adding the abstract keyword to a method removes its body entirely -- just the signature, ending in a semicolon.

Vehicle.java — before

```
public class Vehicle {  
    ...  
    public double monthlyPremium(int milesPerYear) {  
        return 0.0;  
    }  
    ...  
}
```

Vehicle.java — after

```
public abstract class Vehicle {  
    ...  
    public abstract double monthlyPremium(int milesPerYear);  
    ...  
}
```

abstract double monthlyPremium(int milesPerYear); has no body and no braces — it's a promise every concrete subclass must keep, checked at compile time.

Nobody ever wanted the \$0.00 default -- abstract turns that into a rule, not a habit

A plain, generic Vehicle policy was never real. The concrete version on the previous slide's left side didn't stop anyone from writing new `Vehicle("V-999", "Unknown")` and silently billing it at \$0.00 forever -- nothing enforced that every real subclass actually override it.

- ✓ **Ties back to encapsulation**
303.7's encapsulation hides HOW a field is stored. Abstraction hides (and here, mandates) HOW a method is implemented -- both exist so callers depend on a promise, not on one class's specific internals.

An abstract method converts "every subclass is supposed to override this" from a comment into something the compiler actually checks.

Now new Vehicle(...) is a compile error, everywhere

The moment `monthlyPremium()` has no body, the class itself can no longer be built.

ABSTRACTCANTINstantiate.JAVA — DOES NOT COMPILE

```
public class AbstractCantInstantiate {  
    public static void main(String[] args) {  
        Vehicle v = new Vehicle("V-999", "Unknown");  
        System.out.println(v);  
    }  
}
```

JAVAC OUTPUT

```
AbstractCantInstantiate.java:3: error:  
Vehicle is abstract; cannot be instantiated  
Vehicle v = new Vehicle("V-999", "Unknown");  
1 error
```

Sedan, Truck, and Motorcycle are all still buildable -- each one implements `monthlyPremium()`, which is exactly what makes them concrete.

An abstract class can't be missing pieces at runtime. If a piece is missing, the build fails before the program ever gets a chance to run.

The rules abstract classes actually follow

RULE	WHAT IT MEANS HERE
Cannot be instantiated	new Vehicle(...) never compiles, anywhere
Zero or more abstract methods	Vehicle has exactly one — monthlyPremium(). getModel(), vehicleType(), and toString() still have real bodies
Can have fields, constructors, and concrete methods	vin and model are still stored and still set through a constructor
Subclass must implement every abstract method, or be abstract itself	Sedan, Truck, and Motorcycle all implement monthlyPremium() — that's why they're instantiable and Vehicle isn't
An abstract method can't be private or final	private would be invisible to the subclass required to override it; final can never be overridden at all -- both would make the promise impossible to keep

An abstract class is a partly-built parent: some behavior is shared and finished, and some is deliberately left for every subclass to finish itself.

Use abstract when there's no sensible shared default to fall back on

	CONCRETE PARENT (LIKE GETMODEL())	ABSTRACT METHOD (LIKE MONTHLYPREMIUM())
There's a correct shared answer	Yes — every vehicle's getModel() works identically	No — no single number is right for every vehicle type
Risk of leaving it unwritten	None — the shared body is always correct	A silent, wrong default (\$0.00) that nobody notices until billing runs
This lesson uses it for	getModel(), toString() — genuinely identical logic	monthlyPremium() — genuinely different logic per subtype

If every subclass would legitimately write the same body, leave it concrete. If even one subclass needs different logic, make it abstract and let the compiler enforce that every subclass supplies its own.

SECTION FOUR

Interfaces

What it is, what it's for, how to use it, and when. Warehouses aren't vehicles, and this section is why they can still share a coverage board with one.

An interface is a contract: a list of method signatures a class promises to implement

No fields of its own, no constructor, no shared state between the classes that implement it — just a promise that a particular method will exist.

INSURABLE.JAVA

```
public interface Insurable {  
    String policyId();  
  
    default String coverageLevel() {  
        return "STANDARD";  
    }  
}
```

`policyId()` has no body — every implementer must supply one. `coverageLevel()` is a default method: it HAS a body, so implementers get it for free and only override it if they need something different.

An interface says WHAT a class can do. It never says HOW -- that's left entirely to whoever implements it.

Interfaces let classes that share nothing else still be treated the same way

A Sedan and a Warehouse have no common parent and never should -- but the fleet system still needs to bill both the same way.

Vehicle answers "what do Sedan, Truck, and SemiTruck actually have in common?" That question doesn't have an answer for Sedan and Warehouse -- a building genuinely is not a kind of vehicle, and extends Vehicle would be a lie. Insurable asks a smaller, different question instead: can this object report a policy ID and a coverage level? Both a Sedan and a Warehouse can answer yes, with completely different logic underneath.

extends shares an ancestor. implements shares nothing but a promise -- which is exactly enough to put unrelated types in the same array.

Any class implements Insurable by writing its own policyId() -- that's the entire requirement

Warehouse never extends Vehicle — a building isn't a kind of vehicle. It just also implements Insurable.

WAREHOUSE.JAVA

```
public class Warehouse implements Insurable {  
    private final String warehouseId;  
  
    public Warehouse(String warehouseId) { this.warehouseId = warehouseId; }  
  
    @Override  
    public String policyId() { return "WH-" + warehouseId; }  
}
```

INSIDE A MAIN() METHOD

```
Insurable[] things = { new Sedan("V-100", "Camry"), new Warehouse("14") };  
  
for (int i = 0; i < things.length; i++) {  
    System.out.println(  
        things[i].policyId() + " : " + things[i].coverageLevel();  
    }  
}
```

OUTPUT

```
V-100 : STANDARD  
WH-14 : STANDARD
```

A Warehouse could never fit in a Vehicle[] — it isn't one. It fits in an Insurable[] because that's all the loop actually needs from it.

Vehicle implements Insurable too -- the full class, not a fragment

A class can extend one parent AND implement any number of interfaces at the same time. This is Vehicle's complete, final shape.

VEHICLE.JAVA

```
public abstract class Vehicle implements Insurable {
    private final String vin, model;

    protected Vehicle(String vin, String model) {
        this.vin = vin;
        this.model = model;
    }

    public String getModel() { return model; }
    public abstract double monthlyPremium(int milesPerYear);
    public String vehicleType() { return "Vehicle"; }

    @Override
    public String policyId() { return vin; }

    @Override
    public String toString() {
        return vehicleType() + " " + vin + " (" + model + ")";
    }
}
```

implements is a promise about behavior, not a shared parent. Unlike **extends**, which allows only one parent, a class can implement any number of interfaces on top of it.

But that never makes Warehouse a Vehicle

Warehouse implements Insurable without touching the class hierarchy at all -- and the compiler enforces exactly that distinction.

FLEETONLY.JAVA — DOES NOT COMPILE

```
public class FleetOnly {  
    public static void main(String[] args) {  
        Vehicle[] fleet = { new Sedan("V-100", "Camry"), new Warehouse("14") };  
        System.out.println(fleet.length);  
    }  
}
```

JAVAC OUTPUT

```
FleetOnly.java:3: error: incompatible types:  
Warehouse cannot be converted to Vehicle  
Vehicle[] fleet = { new Sedan("V-100", "Camry"), new Warehouse("14") };  
1 error
```

implements adds a promise, not a place in the family tree. Warehouse keeps every one of Insurable's promises and still isn't, and never becomes, a Vehicle.

abstract class vs interface: two different sharing problems

	ABSTRACT CLASS	INTERFACE
Shares	state (fields) and behavior (method bodies)	only a method signature, plus optional default bodies
A class can have	one parent	any number of interfaces
Use it when	types are genuinely related and share real data	unrelated types need to promise the same capability
In this lesson	Vehicle — vin, model, and toString() are actually shared	Insurable — Sedan and Warehouse share nothing but a method name

Vehicle answers "what do these types have in common?" Insurable answers "what can these unrelated types both do?" Different questions, different tools.

YOU TRY IT

Give SemiTruck its own coverage level

YOUR TURN · 4 MIN

SOLVE THIS

Commercial semi trucks need different coverage than the default. Override `coverageLevel()` on `SemiTruck` to return "COMMERCIAL" instead of using the interface's default.

One way to write it

ONE WAY TO DO IT

SEMITRUCK.JAVA (EXCERPT) — ONE NEW OVERRIDE

```
public class SemiTruck extends Truck {  
    ...  
    @Override  
    public String coverageLevel() { return "COMMERCIAL"; }  
}
```

```
NEW SEMITRUCK(...).COVERAGELEVEL()
```

```
COMMERCIAL
```

SemiTruck inherits implements Insurable from Vehicle — this override works the same way monthlyPremium()'s did.

Three types, three different bodies, one Insurable[] loop -- zero type-checking

Sedan and Warehouse share nothing but the interface. SemiTruck's override from the practice above runs right alongside them, untouched.

INSIDE A MAIN() METHOD

```
Insurable[] insured = {  
    new Sedan("V-100", "Camry"),  
    new Warehouse("14"),  
    new SemiTruck("V-201", "Freightliner Cascadia")  
};  
  
for (int i = 0; i < insured.length; i++) {  
    System.out.println(  
        insured[i].policyId() + " : " + insured[i].coverageLevel();  
    )  
}
```

OUTPUT

```
V-100 : STANDARD  
WH-14 : STANDARD  
V-201 : COMMERCIAL
```

Sedan and Warehouse never wrote coverageLevel() -- they get Insurable's default. SemiTruck did, on the slide right before this one, and the loop picks that up automatically. Nobody checked what type index i was before calling either method.

This is the payoff of implements: three types with nothing else in common, one loop, and every override still runs exactly where it should.

IS-A shares an ancestor; HAS-A just holds a reference

Both are ways to reuse code. They answer different questions about the relationship.

	INHERITANCE (IS-A)	COMPOSITION (HAS-A)
Relationship	SemiTruck IS-A Truck	A fleet system HAS-A list of Insurable things
Coupling	Tight — subclass depends on parent's internals staying stable	Loose — the holder only depends on the held object's public methods
Change risk	Changing Vehicle can ripple into every subclass	Swapping which Insurable is in the array affects nothing else
This lesson uses	extends for Sedan/Truck/SemiTruck — they really are kinds of Vehicle	the Insurable[] array in the contract demo — it holds things, it isn't one

Prefer composition when you're tempted to inherit just to reuse a method. Reserve extends for a genuine IS-A.

Why this matters past this lesson

Every one of today's tools solves the same underlying problem: code that has to handle several related things without an if/else chain that grows every time a new one shows up.



The test that ties it together

Could you add a fifth vehicle type -- say, an SUV -- without editing PremiumReport, the fleet board, or the coverage demo at all? If the answer is yes, polymorphism is doing its job.

That's also exactly what the guided build in the next section checks for.

Inheritance and interfaces aren't separate topics from polymorphism — they're what make polymorphism possible in the first place.

SECTION FIVE

Guided Build — The Fleet Coverage Board

Everything from today, assembled into one small system, decision by decision.

The ticket

FLEET-58 One board and one monthly billing report for everything insurable

5 pts

The fleet system currently tracks sedans, trucks, and warehouse policies in three separate places. Replace that with one board and one report.

ACCEPTANCE CRITERIA

1. Sedans and trucks bill monthly premiums on their own mileage tiers; semi trucks add a flat \$50.00 commercial surcharge on top of the truck tiers
2. Every insurable thing — vehicles AND warehouses — can appear on one coverage board showing its policy ID and its coverage level
3. A monthly billing report totals premiums across every insured vehicle
4. Adding a new vehicle type must never require editing the board loop or the report loop
5. Warehouses must appear on the same coverage board as vehicles, even though a warehouse is not a vehicle

Two different sharing problems, two different tools

WHAT WE NEED	A way to share vin/model storage and default behavior across vehicle types, AND a way to put vehicles and warehouses on the same board.
OPTIONS	One shared parent class handles state + behavior. One shared interface handles a capability across unrelated types. They solve different halves of this ticket.
WE PICK	abstract class Vehicle for the vehicle family; interface Insurable for the board
BECAUSE	Vehicle gives Sedan, Truck, and SemiTruck their state and default toString(). Insurable is the only thing a Warehouse and a Sedan will ever have in common.

- 1 Insurable interface
- 2 Vehicle abstract class
- 3 Sedan and Truck
- 4 SemiTruck and Warehouse
- 5 FleetBoard
- 6 MonthlyBilling

Step 1 — the contract and the shared parent

STEP 1 OF 6 · THE CONTRACT AND THE SHARED PARENT

INSURABLE.JAVA

```
public interface Insurable {  
    String policyId();  
    default String coverageLevel() { return "STANDARD"; }  
}
```

VEHICLE.JAVA

```
public abstract class Vehicle implements Insurable {  
    private final String vin, model;  
  
    protected Vehicle(String vin, String model) {  
        this.vin = vin;  
        this.model = model;  
    }  
  
    public String getModel() { return model; }  
    public abstract double monthlyPremium(int milesPerYear);  
    public String vehicleType() { return "Vehicle"; }  
  
    @Override  
    public String policyId() { return vin; }  
  
    @Override  
    public String toString() {  
        return vehicleType() + " " + vin + " (" + model + ")";  
    }  
}
```

Step 2 — the two ordinary vehicle types

STEP 2 OF 6 · THE TWO ORDINARY VEHICLE TYPES

SEDAN.JAVA

```
public class Sedan extends Vehicle {
    public Sedan(String vin, String model) { super(vin, model); }

    @Override
    public double monthlyPremium(int milesPerYear) {
        if (milesPerYear < 5000) return 45.00;
        if (milesPerYear < 15000) return 65.00;
        if (milesPerYear < 25000) return 90.00;
        return 120.00;
    }

    @Override
    public String vehicleType() { return "Sedan"; }
}
```

TRUCK.JAVA

```
public class Truck extends Vehicle {
    public Truck(String vin, String model) { super(vin, model); }

    @Override
    public double monthlyPremium(int milesPerYear) {
        if (milesPerYear < 5000) return 70.00;
        if (milesPerYear < 15000) return 95.00;
        if (milesPerYear < 25000) return 130.00;
        return 170.00;
    }

    @Override
    public String vehicleType() { return "Truck"; }
}
```

Step 3 — the odd ones out

SemiTruck extends the family. Warehouse refuses to.

STEP 3 OF 6 · THE ODD ONES OUT

SEMITRUCK.JAVA

```
public class SemiTruck extends Truck {
    public SemiTruck(String vin, String model) { super(vin, model); }

    @Override
    public double monthlyPremium(int milesPerYear) {
        return super.monthlyPremium(milesPerYear) + 50.00;
    }

    @Override
    public String vehicleType() { return "SemiTruck"; }

    @Override
    public String coverageLevel() { return "COMMERCIAL"; }
}
```

WAREHOUSE.JAVA

```
public class Warehouse implements Insurable {
    private final String warehouseId;

    public Warehouse(String warehouseId) { this.warehouseId = warehouseId; }

    @Override
    public String policyId() { return "WH-" + warehouseId; }
}
```

Step 4 — the board

Criterion 5 in one loop: a Sedan, a SemiTruck, and a Warehouse on the same array.

STEP 4 OF 6 · THE BOARD

FLEETBOARD.JAVA — INSIDE MAIN()

```
Insurable[] board = {  
    new Sedan("V-100", "Camry"),  
    new SemiTruck("V-201", "Freightliner Cascadia"),  
    new Warehouse("14")  
};  
  
for (int i = 0; i < board.length; i++) {  
    System.out.println(  
        board[i].policyId() + " : " + board[i].coverageLevel());  
}
```

OUTPUT

```
V-100 : STANDARD  
V-201 : COMMERCIAL  
WH-14 : STANDARD
```

board is an `Insurable[]`, not a `Vehicle[]` — that's the only array type a Warehouse can sit in next to a Sedan.

Step 5 — the monthly billing report

Criteria 1 and 3: premiums by tier, totaled, with zero type-checking in the loop.

STEP 5 OF 6 · THE MONTHLY BILLING REPORT

MONTHLYBILLING.JAVA — INSIDE MAIN()

```
Vehicle[] fleet = {  
    new Sedan("V-210", "Malibu"),  
    new Truck("V-330", "Silverado 2500"),  
    new SemiTruck("V-331", "Peterbilt 579"),  
    new Sedan("V-211", "Accord"),  
    new Truck("V-332", "Ram 1500")  
};  
  
double totalPremiums = 0.0;  
for (int i = 0; i < fleet.length; i++) {  
    double premium = fleet[i].monthlyPremium(18000);  
    System.out.println(fleet[i] + " $" + premium);  
    totalPremiums = totalPremiums + premium;  
}  
  
System.out.println("Monthly total: $" + totalPremiums);
```

OUTPUT

```
Sedan V-210 (Malibu) $90.0  
Truck V-330 (Silverado 2500) $130.0  
SemiTruck V-331 (Peterbilt 579) $180.0  
Sedan V-211 (Accord) $90.0  
Truck V-332 (Ram 1500) $130.0  
Monthly total: $620.0
```

TICKET CLOSED

Every acceptance criterion, traced to the line that satisfies it

**SEDANS/TRUCKS
BILL BY TIER,
SEMIS ADD \$50.00**

Sedan.monthlyPremium() / Truck.monthlyPremium() / SemiTruck.monthlyPremium() calling super.monthlyPremium(...)

**EVERYTHING
INSURABLE
APPEARS ON ONE
BOARD**

FleetBoard's Insurable[] board, mixing Vehicle subtypes with Warehouse

**MONTHLY REPORT
TOTALS EVERY
INSURED
VEHICLE'S
PREMIUM**

MonthlyBilling's Vehicle[] loop and running totalPremiums

**A NEW VEHICLE
TYPE NEEDS NO
LOOP EDITS**

neither loop branches on type — both just call the interface/abstract method and let overriding pick the behavior

**WAREHOUSES
SHARE THE BOARD
DESPITE NOT
BEING VEHICLES**

Warehouse implements Insurable directly; it was never a Vehicle and didn't need to be

What you can defend after today

extends

IS-A. One parent, `super()` first, and `@Override` on every replacement

Two types

declared sets the menu and is checked by the compiler; runtime cooks the meal

abstract

a type with no sensible instance, and a method the compiler forces subclasses to write

interface

a capability contract across unrelated types, with default methods for shared bodies

Next: collections — where `List` is an interface, `ArrayList` is an implementation, and today's lesson is why that sentence matters.